A Project Report

On

SOCIAL NETWORK ANALYSIS

BY

Darshan Chandak

2020A7PS2085H

Under the supervision of

Dr. Apurba Das

SUBMITTED IN COMPLETE FULFILLMENT OF THE REQUIREMENTS OF

CS F376: DESIGN PROJECT

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI (RAJASTHAN)

HYDERABAD CAMPUS

(May 2023)

ACKNOWLEDGMENTS

I am indebted to Dr. Apurba Das, my project mentor, for his constant guidance and feedback on my work.

I sincerely thank the Department of Computer Science and Information Systems (CSIS), BITS Pilani Hyderabad Campus, for providing me with this excellent opportunity to gain valuable insights and learn new things via this Formal Design Project.

A big thanks to Dr. Rajib Ranjan Maiti, whose guidance has gone a long way toward the successful execution of the project so far.

I am indebted to my parents, teachers, and friends, as without their prayers and best wishes, I wouldn't have reached this far.

Birla Institute of Technology and Science-Pilani,

Hyderabad Campus

CERTIFICATE

This is to certify that the project report entitled "**SOCIAL NETWORK ANALYSIS"** submitted by **Mr. Darshan Chandak** (ID No. **2020A7PS2085H**) in complete fulfillment of the requirements of the course **CS F376, Design Project Course**, embodies the work done by him under my supervision and guidance.

Date: 19/05/2023 **(Dr. Apurba Das)**

BITS- Pilani, Hyderabad Campus

ABSTRACT

Missing data in time series data is a pervasive problem that puts obstacles in the way of advanced data analysis. A popular solution is data imputation, where the fundamental challenge is determining what values should be filled in, in the place of the missing values.

In this project, a Time Series Data Imputation has been successfully been performed for Geolife Trajectory Dataset. Morever, data is being analyzed for each individual user by converting the latitude and longitude data into x and y coordinates. Then these x and y points are plotted on a graph, and then a trajectory graph for a user is obtained for a particular day.

A codebase is formed where we can input the user number and the start and end date for that particular user and get in resultant the imputed data for that user, and corresponding x and y coordinates. We can get data for the user's longitude and latitude values for the particular day in a frequency of 30 minutes, even if this particular time's value is missing in the original dataset.

We are using SAITS: Self-Attention-based Imputation for Time Series model by training it on our geolife trajectory dataset for Time Series Data Imputation.

CONTENTS

TITLE PAGE 1

ACKNOWLEDGEMENTS 2

CERTIFICATE 3

ABSTRACT 4

INTRODUCTION 6

1. Time Series Data 6
2. SAITS 7
3. Geolife GPS Trajectory Dataset 8

DATA FORMATS 9

1. Trajectory File 9
2. Transportation Mode Labels File 10

DATA IMPUTATION METHODOLOGY 11

1. Data Pre-processing 11
2. Generating Required Dataframes 12
3. SAITS Model Training 14
4. Data Imputation 17
5. Some other Plots 19

FURTHER PROSPECTS 21

TRAJECTORY SIMILARITY MEASURES 22

Conclusion 23

References 24

INTRODUCTION

1. Time Series Data

Time series data is a collection of observations acquired through repeated measurements over time. Plot the points on a graph; one of your axes would always be time.
Time series metrics refer to data followed at a specific increment in time.
For example, a metric could refer to how much stock is sold in a store from one day to another.

Time series data is around since time is a component of everything observable. As our world gets increasingly instrumented, sensors and systems constantly emit a persistent stream of time series data. Such data has numerous applications across various industries.

Let us put this in context through some examples.

Examples of time series analysis:

- Rainfall measurements
- Stock prices
- Annual retail sales
- Monthly subscribers
- Heartbeats per minute

2. SAITS

SAITS: Self-Attention-based Imputation for Time Series

Motivation: SAITS is developed mainly to help confound the defects (slow speed, memory limitations, and compounding error) of RNN(Recurrent Neural Networks) -based imputation models and to obtain state-of-the-art (SOTA) imputation accuracy on partially-observed time series.

Performance: SAITS outdoes BRITS(Bidirectional Recurrent Imputation for Time Series) by 12% to 38% in MAE (Mean Absolute Error) and achieves 2.0 to 2.6 times faster training speed. Also, SAITS outperforms Transformer by 2% to 19% in MAE with a more efficient model structure (to obtain similar performance, SAITS needs only 15% to 30% parameters of Transformer). Compared to another SOTA self-attention imputation model NRTSI(Non-Recurrent Time Series Imputation), SAITS achieves a 7% to 39% smaller Mean Squared Error (above 20% in nine out of sixteen cases). Meanwhile, it requires much fewer parameters and less imputation time in practice.

Methodology:

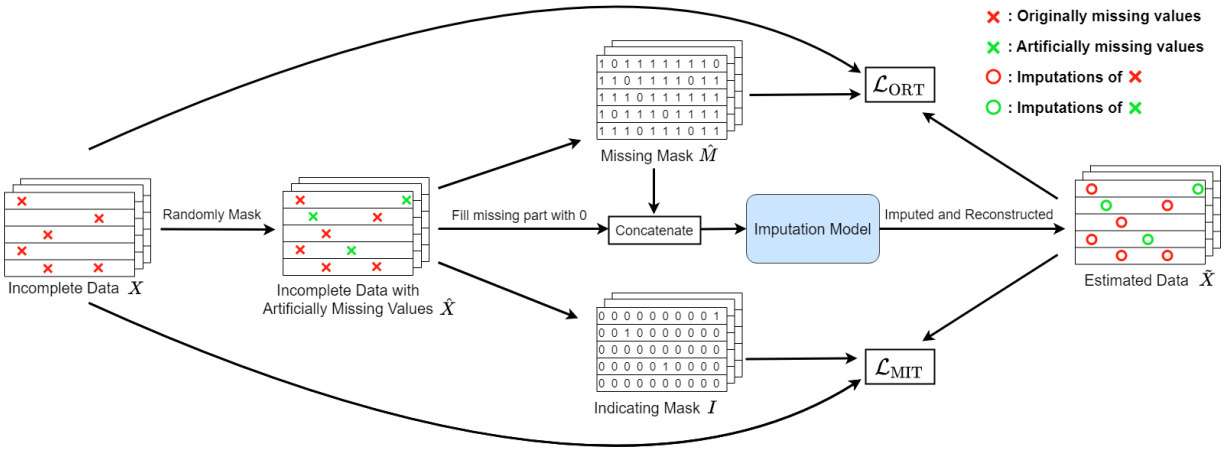


Figure 1: A graphical overview of the joint-optimization training approach, illustrating the implementation details of the two training tasks: Masked Imputation Task (MIT) and Observed Reconstruction Task (ORT).

GitHub Link: https://github.com/WenjieDu/SAITS
Official Research Paper Link: https://arxiv.org/pdf/2202.08516.pdf

3. Geolife GPS Trajectory Dataset

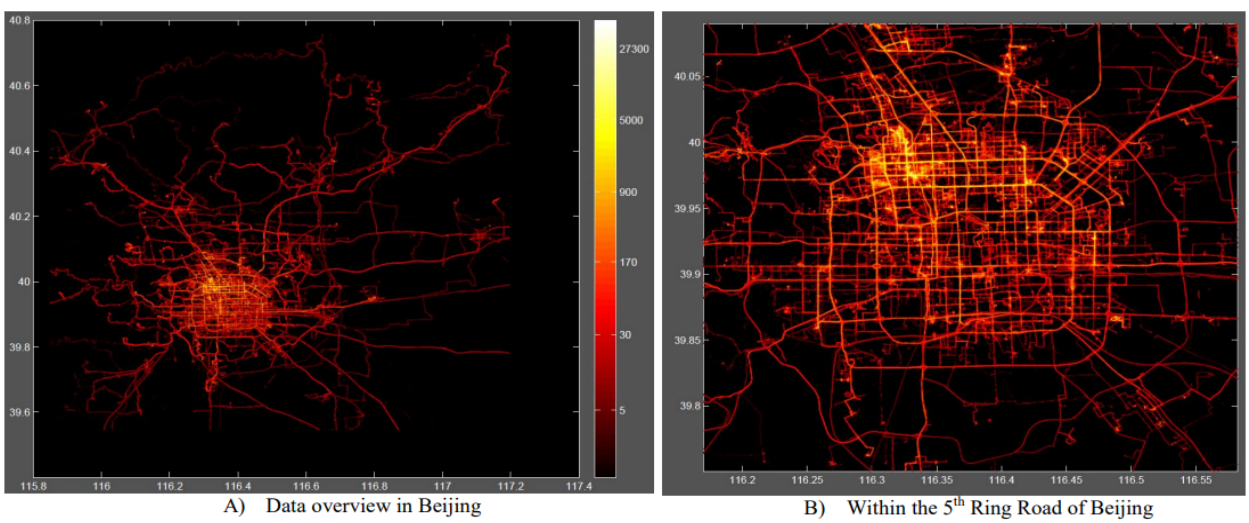
This GPS trajectory dataset was collected in the (Microsoft Research Asia) Geolife project by 182 users over five years (from April 2007 to August 2012).
A GPS trajectory of this dataset is represented by a series of time-stamped points, each containing the details of latitude, longitude, and altitude values. This dataset includes 17,621 trajectories with a total distance of 1,292,951 kilometers and a total duration of 50,176 hours.

These trajectories were registered at a variety of sampling rates. 91.5 percent of the trajectories are logged in a dense representation, e.g., every 1 to 5 seconds or every 5 to 10 meters per point. This dataset documented a wide range of users' outdoor activities, including life routines like going home and work and recreation and sports activities like shopping, sightseeing, dining, hiking, and cycling.

This trajectory dataset can be utilized in many research fields, such as mobility pattern mining, user activity recognition, location-based social networks, location privacy, and location recommendation.

Although this dataset is widely dispersed in over 30 cities in China and even in some cities in the USA and Europe, most of the data was generated in Beijing, China.

Figure 2 plots this dataset's distribution (heat map) in Beijing. The statistics on the right side of the heat bar denote the number of points generated in a location.



A) Data overview in Beijing B) Within the 5th Ring Road of Beijing

Figure 2: Distribution of the dataset in Beijing city

Dataset Link: [Geolife GPS trajectory dataset - User Guide - Microsoft Research]

DATA FORMATS

1. Trajectory File

Every folder of this dataset stores a user's GPS log files, which were transformed into PLT format. Each PLT file contains a single trajectory and is named by its starting time. We use GMT in each point's date/time property to avoid possible time zone confusion.

PLT format:

Lines 1…6 are useless in this dataset and can be ignored. Points are defined in the following lines, one for each line.

Field 1: Latitude in decimal degrees.

Field 2: Longitude in decimal degrees.

Field 3: All set to 0 for this dataset.

Field 4: Altitude in feet.

Field 5: Date - number of days (with a fractional part) that have passed since 12/30/1899.

Field 6: Date as a string.

Field 7: Time as a string.

Note that Field 5 and Field 6 & 7 represent the same date/time in this dataset. We may use either of them.

Example:

39.906631,116.385564,0,492,40097.5864583333,2009-10-11,14:04:30
39.906554,116.385625,0,492,40097.5865162037,2009-10-11,14:04:35

In our code, we drop the initial six lines and Fields 3, 4, and 5 in the data pre-processing section.

2. **Transportation Mode Labels File**

Some users might have an optional **labels.txt file** in the corresponding folder. It has details about the transportation mode of the user.

Probable transportation modes are: walk, bike, bus, car, subway, train, airplane, boat, run, and motorcycle.

Also, we have transformed the date/time of all labels to GMT, even though most of them were generated in China.

Example:

Start Time	End Time	Transportation Mode
2008/04/02 11:24:21	2008/04/02 11:50:45	bus
2008/04/03 01:07:03	2008/04/03 11:31:55	train
2008/04/03 11:32:24	2008/04/03 11:46:14	walk
2008/04/03 11:47:14	2008/04/03 11:55:07	car

DATA IMPUTATION METHODOLOGY

1. Data Pre-processing

Here, we read the entire dataset folder (Data Folder).

The folder structure of the dataset is as follows:

The folder has 182 subfolders corresponding to each user. Then each subfolder has a sub-sub folder named "Trajectory." This Trajectory folder has all the .plt files corresponding to the user's GPS Trajectory Data. It can also have an optional file labeled "labels.txt." This .txt file contains information about the transportation mode the user has used to travel from one location to another. Each .plt file is named after the starting time of the first data point.

DATA PRE-PROCESSING

```python
import numpy as np
import pandas as pd
import glob
import os.path
import datetime
import os
```

[1] ✓ 1.4s Python

```python
def read_plt(plt_file):
    points = pd.read_csv(plt_file, skiprows=6, header=None, parse_dates=[[5, 6]], infer_datetime_format=True)

    # for clarity rename columns
    points.rename(inplace=True, columns={'5_6': 'DateTime', 0: 'Latitude', 1: 'Longitude'})

    # remove unused columns
    points.drop(inplace=True, columns=[2, 3, 4])

    return points
```

[2] ✓ 0.0s Python

We are dropping the columns - 2, 3, 4, i.e., field values corresponding to 3, 4, 5 (Altitude, Zero Value, Fractional DateTime Value).

```python
mode_names = ['walk', 'bike', 'bus', 'car', 'subway','train', 'airplane', 'boat', 'run', 'motorcycle', 'taxi']
mode_ids = {s : i + 1 for i, s in enumerate(mode_names)}
```

[3] ✓ 0.0s Python

```python
def read_labels(labels_file):
    labels = pd.read_csv(labels_file, skiprows=1, header=None, parse_dates=[[0, 1], [2, 3]], infer_datetime_format=True, delim_whitespace=Tru

    # for clarity rename columns
    labels.columns = ['StartTime', 'EndTime', 'Label']

    # replace 'Label' column with integer encoding
    labels['Label'] = [mode_ids[i] for i in labels['Label']]

    return labels
```

[4] ✓ 0.0s Python

```python
def apply_labels(points, labels):
    indices = labels['StartTime'].searchsorted(points['DateTime'], side='right') - 1
    no_label = (indices < 0) | (points['DateTime'].values >= labels['EndTime'].iloc[indices].values)
    points['Label'] = labels['Label'].iloc[indices].values
    points['Label'][no_label] = 0
```

[5] ✓ 0.0s Python

Then we apply labels wherever possible. We assign each transportation mode a specific number, like walk-1, bike-2, bus-3, and so on. We apply a label as 0 when there is no label file present.

```python
def read_user(user_folder):
    labels = None

    plt_files = glob.glob(os.path.join(user_folder, 'Trajectory', '*.plt'))
    df = pd.concat([read_plt(f) for f in plt_files])

    labels_file = os.path.join(user_folder, 'labels.txt')
    if os.path.exists(labels_file):
        labels = read_labels(labels_file)
        apply_labels(df, labels)
    else:
        df['Label'] = 0

    return df
```

[6] ✓ 0.0s Python

```python
def read_all_users(folder):
    subfolders = os.listdir(folder)
    dfs = []
    for i, sf in enumerate(subfolders):
        print('[%d/%d] Processing User %s' % (i + 1, len(subfolders), sf))
        df = read_user(os.path.join(folder,sf))
        df['User'] = int(sf)
        dfs.append(df)
    return pd.concat(dfs)
```

[7] ✓ 0.0s Python

We then iterate through all users for all days to read all the present data.

2. Generating Required Dataframes

GENERATING REQUIRED DATAFRAMES

```python
df = read_all_users('Data')
```

[8] ✓ 3m 14.9s Python

```
[1/182] Processing User 000
[2/182] Processing User 001
[3/182] Processing User 002
[4/182] Processing User 003
[5/182] Processing User 004
[6/182] Processing User 005
[7/182] Processing User 006
[8/182] Processing User 007
[9/182] Processing User 008
[10/182] Processing User 009
[11/182] Processing User 010
```

We are processing the 182 users.

```python
df_user = []
for i in range(182):
    df_user.append(df[df["User"] == i])
df_user
```

[10] ✓ 10.3s Python

```
Output exceeds the size limit. Open the full output data in a text editor
[                 DateTime   Latitude   Longitude  Label  User
 0    2008-10-23 02:53:04  39.984702  116.318417      0     0
 1    2008-10-23 02:53:10  39.984683  116.318450      0     0
 2    2008-10-23 02:53:15  39.984686  116.318417      0     0
 3    2008-10-23 02:53:20  39.984688  116.318385      0     0
 4    2008-10-23 02:53:25  39.984655  116.318263      0     0
 ...                  ...        ...        ...    ...   ...
 2307 2009-07-05 07:44:55  40.000403  116.327255      0     0
 2308 2009-07-05 07:45:00  40.000433  116.327209      0     0
 2309 2009-07-05 07:45:05  40.000443  116.327186      0     0
 2310 2009-07-05 07:45:10  40.000522  116.327132      0     0
 2311 2009-07-05 07:45:15  40.000543  116.327148      0     0

 [173870 rows x 5 columns],
```

Separating data for different users.

```python
from datetime import datetime, timedelta
df_merged_user = []
for i in range(182):
    df_merged = []
    for j in range(df_user[i]["DateTime"].dt.date.unique().size):
        date_today = datetime(df_user_dates[i][j].year,df_user_dates[i][j].month,df_user_dates[i][j].day, 0, 0, 0, 0)
        days = pd.date_range(date_today, date_today + timedelta(1), freq='30min')
        df1 = pd.DataFrame({'DateTime': days})
        df1.drop(df1.index[-1],inplace = True)
        df2 = pd.DataFrame(np.nan, index=range(48), columns=['Latitude','Longitude'])
        df_merged.append(df1.join(df2))
    df_merged_user.append(df_merged)
df_merged_user
```

[14] ✓ 48.8s

```
Output exceeds the size limit. Open the full output data in a text editor
[[              DateTime  Latitude  Longitude
  0  2008-10-23 00:00:00       NaN        NaN
  1  2008-10-23 00:30:00       NaN        NaN
  2  2008-10-23 01:00:00       NaN        NaN
  3  2008-10-23 01:30:00       NaN        NaN
  4  2008-10-23 02:00:00       NaN        NaN
  5  2008-10-23 02:30:00       NaN        NaN
  6  2008-10-23 03:00:00       NaN        NaN
  7  2008-10-23 03:30:00       NaN        NaN
  8  2008-10-23 04:00:00       NaN        NaN
  9  2008-10-23 04:30:00       NaN        NaN
  10 2008-10-23 05:00:00       NaN        NaN
  11 2008-10-23 05:30:00       NaN        NaN
```

Now as we want to impute data for all users, we will try to impute data for the user every 30 minutes. So we create a data frame with NaN values for Latitude and Longitude every 30 minutes daily, i.e., 48 imputed data points.

```python
for i in range(182):
    for j in range(df_user[i]["DateTime"].dt.date.unique().size):
        df_user[i] = df_user[i].append(df_merged_user[i][j])
    df_user[i].sort_values(by = ['DateTime'], ascending = True, inplace=True)
    df_user[i] = df_user[i].reset_index(drop=True)
df_user
```

[13] ✓ 2m 47.7s Python

We now merge this data frame with the original data frame for each user.

3. SAITS Model Training

We now have to train the SAITS Model that would be used to impute missing data values in our trajectory dataset.

```
def impute(u, ds, de):
    user = u
    day_start = ds
    day_end = de

    start_datetime = df_user_dates[user][day_start]
    dt = df_user_dates[user][day_end - 1]
    end_datetime = dt + timedelta(1)
    df_temp = pd.DataFrame(df_user[user].query('DateTime >= @start_datetime and DateTime < @end_datetime'))
    print('User: %d' % (user))
    print('Start DateTime: ')
    print(start_datetime)
    print('End DateTime: ')
    print(end_datetime)
    print()
    lat_mean = df_temp['Latitude'].mean()
    print('Latitude Mean: %f' % (lat_mean))
    lon_mean = df_temp['Longitude'].mean()
    print('Longitude Mean: %f' % (lon_mean))
    lat_stdev = np.nanstd(df_temp['Latitude'])
    print('Latitude Standard Deviation: %f' % (lat_stdev))
    lon_stdev = np.nanstd(df_temp['Longitude'])
    print('Longitude Standard Deviation: %f' % (lon_stdev))
    print()
```

Initially, we take 3 parameters as input: the user number, start day index, and end day index. Then we calculate the Latitude Mean, Longitude Mean, Latitude Standard Deviation, and Longitude Standard Deviation.

```
print('MODEL TRAINING')

X = df_temp
num_samples = len(X)
X = X.drop('Label', axis = 1)
X = X.drop('User', axis = 1)
X = X.drop('DateTime', axis = 1)
X = StandardScaler().fit_transform(X.to_numpy())
X = X.reshape(1, num_samples, -1)
X_intact, X, missing_mask, indicating_mask = mcar(X, 0.1) # hold out 10% observed values as ground truth
X = masked_fill(X, 1 - missing_mask, np.nan)
saits = SAITS(n_steps=num_samples, n_features=2, n_layers=5, d_model=256, d_inner=512, n_head=8, d_k=32, d_v=32, dropout=0, epochs=1000)
saits.fit(X)
imputation = (saits.impute(X))
mae = cal_mae(imputation, X_intact, indicating_mask)  # calculate mean absolute error on the ground truth (artificially-missing values)
print()
print('Mean Absolute Error (MAE): %f' % (mae))
```

This is where we train our actual model for imputing the missing data. We standardize the data first to ensure that the data is evenly centered around the mean. So the mean is 0, and the standard deviation is 1.

$$Z = \frac{x - \mu}{\sigma}$$

We hold 10% of the training data or observed values as ground truth or validation datasets. We do this by artificially masking those values.

We use multiple parameters of the SAITS model.
n_steps: number of time steps in the dataset(num_samples = length of the target data frame)
n_features: 2(Latitude and Longitude)
n_layers: number of stacked layers of the diagonally-masked multi-head attention and the feed-forward network
d_model: the dimension of the model
d_inner: the dimension of the feed-forward network
n_head: number of heads
d_k: the dimension of the Query vector
d_v: the dimension of the Value vector
dropout: fraction of the neurons to be zeroed out at each training step
epochs: the total number of iterations of all the training data in one cycle for training the model.

For self-attention models, we sample the n_layers (N) from (1, 2, 3, 4, 5, 6, 7, 8), d_model from (64, 128, 256, 512, 1024), d_inner from (128, 256, 512, 1024, 2048, 4096), d_v from (32, 64, 128, 256, 512), the n_head from (2, 4, 8). d_k is set as the value of d_model divided by n_head.

We have used the following parameters:

```
n_steps = num_samples,
n_features = 2,
n_layers = 5,
d_model = 256,
d_inner = 512,
n_head = 8,
d_k = 32,
d_v = 32,
dropout = 0,
epochs = 1000
```

```
df_user_imputation = pd.DataFrame(imputation.reshape(int(imputation.size / 2), 2), columns=['Latitude','Longitude'])
df_user_final = df_user_imputation.join(df_user_time[user]['DateTime'])
df_user_final = df_user_final[['DateTime','Latitude','Longitude']]
# pd.set_option('display.max_rows', None)
for i in range(df_user_final.shape[0]):
    df_user_final['Latitude'].iloc[i] = (df_user_final['Latitude'].iloc[i] * lat_stdev) + lat_mean
    df_user_final['Longitude'].iloc[i] = (df_user_final['Longitude'].iloc[i] * lon_stdev) + lon_mean
```

Now that we have standardized the data for imputation, we need to convert it back to the original form. So we multiply the standard deviation value and add the mean value which we calculated earlier.

```
xcord = []
ycord = []
for i in range(len(df_user_final)):
    x, y, zone, ut = utm.from_latlon(df_user_final['Latitude'].iloc[i], df_user_final['Longitude'].iloc[i])
    xcord.append(x)
    ycord.append(y)
columns=['DateTime','Latitude_X','Longitude_Y']
df_xy = pd.DataFrame(list(zip(df_user_time[user]['DateTime'], xcord, ycord)), columns=columns)

# create a new dataframe with only 30-minute intervals
df_xy_30min = df_xy.set_index('DateTime').resample('30min').first().reset_index()
# use iloc to select only rows with DateTime values that are multiples of 30 minutes
df_xy_30min = df_xy_30min.iloc[::1, :]
```

We now convert the final data points obtained above to x and y coordinates.
To achieve this, we use the **utm** standard library in Python to convert latitude and longitude points to x and y coordinates,

```
print()
print('GRAPH')

fig = plt.figure()
ax = fig.add_subplot(111)
ax.set_xlabel('X co-ordinate')
ax.set_ylabel('Y co-ordinate')
ax.xaxis.label.set_color('red')
ax.yaxis.label.set_color('red')
ax.tick_params(axis='x', colors='white')
ax.tick_params(axis='y', colors='white')
plt.plot(df_xy['Latitude_X'], df_xy['Longitude_Y'],color = 'blue')
plt.plot(df_xy_30min['Latitude_X'], df_xy_30min['Longitude_Y'],color = 'red')
plt.show()
```

Then we plot these coordinates on a graph. We initially plot all the data points (Original + Imputed) with the blue curve and then only the imputed data points with the red curve.

4. Data Imputation

```
User: 0
Start DateTime:
2008-10-23
End DateTime:
2008-10-24

Latitude Mean: 39.998040
Longitude Mean: 116.315561
Latitude Standard Deviation: 0.010484
Longitude Standard Deviation: 0.007876
```

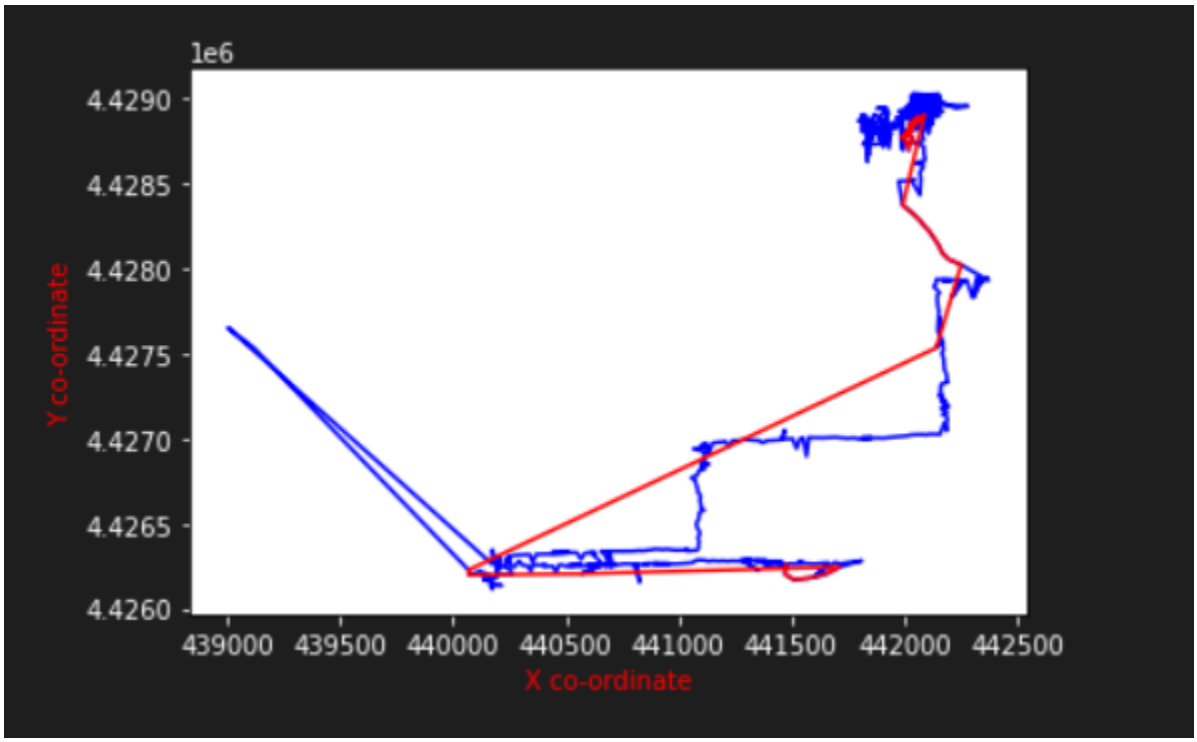
This is the initial user data.

```
MODEL TRAINING
No given device, using default device: cpu
Model initialized successfully. Number of the trainable parameters: 5266312
epoch 0: training loss 1.9188
epoch 1: training loss 7.4765
epoch 2: training loss 1.8881
epoch 3: training loss 2.7339
epoch 4: training loss 2.1780
epoch 5: training loss 1.3154
epoch 6: training loss 2.2782
epoch 7: training loss 2.0744
epoch 8: training loss 0.9362
epoch 9: training loss 1.5332
epoch 10: training loss 1.5709
epoch 11: training loss 1.2029
epoch 12: training loss 0.7988
epoch 13: training loss 1.0774
epoch 14: training loss 1.0647
epoch 15: training loss 0.8402
epoch 16: training loss 0.7421
epoch 17: training loss 0.7992
epoch 18: training loss 0.7566
epoch 19: training loss 0.6815
epoch 20: training loss 0.7097
epoch 21: training loss 0.6867
epoch 22: training loss 0.6235
epoch 23: training loss 0.6146
epoch 24: training loss 0.6062
epoch 25: training loss 0.5871
epoch 26: training loss 0.5946
epoch 27: training loss 0.5951
epoch 28: training loss 0.5769
epoch 29: training loss 0.5672
```

This is the model training. We can clearly see the drop in training loss in consequent epochs.

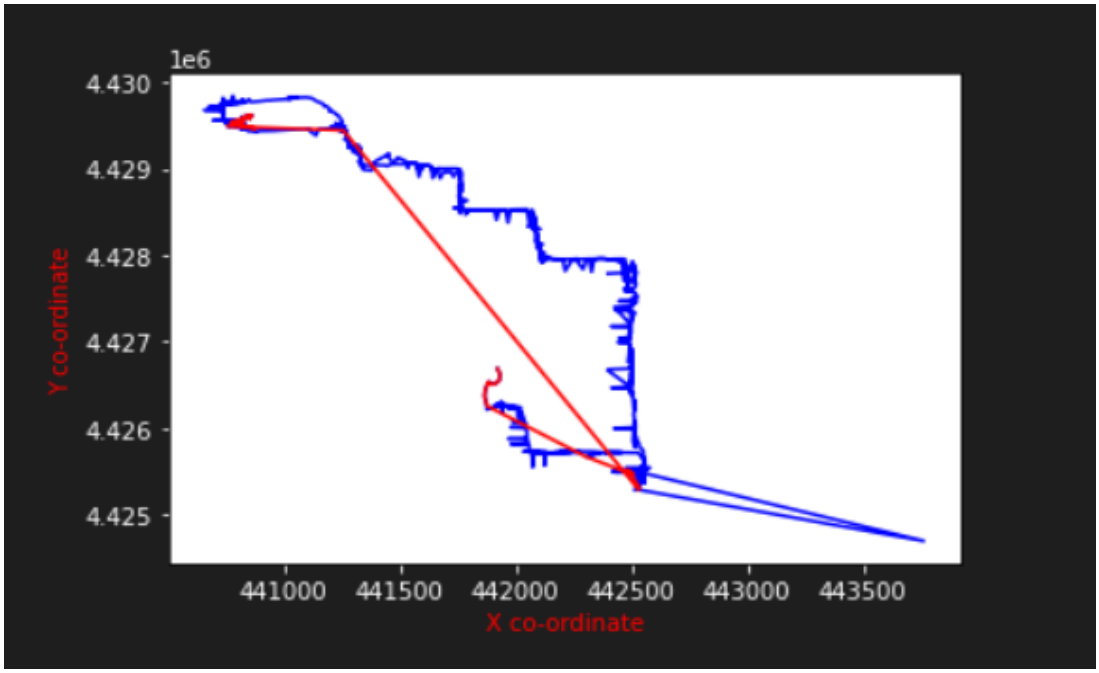
```
73    Mean Absolute Error (MAE): 0.059751
```

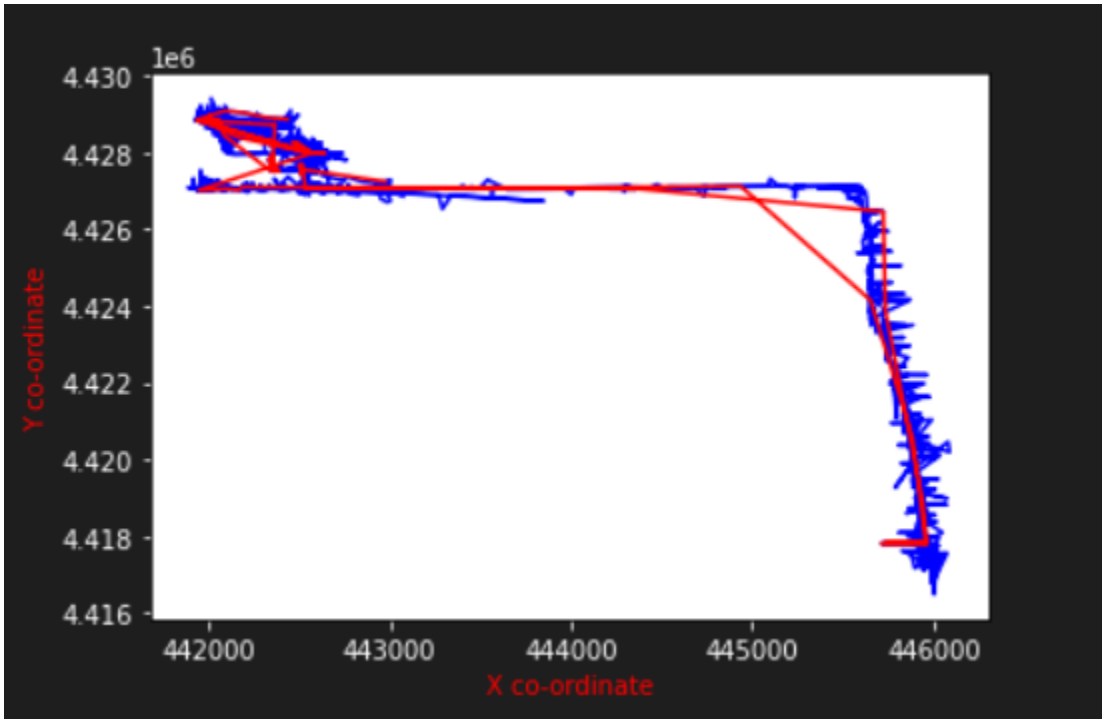
This is the **MEAN ABSOLUTE ERROR** or **MAE** calculated from the ground truth values.

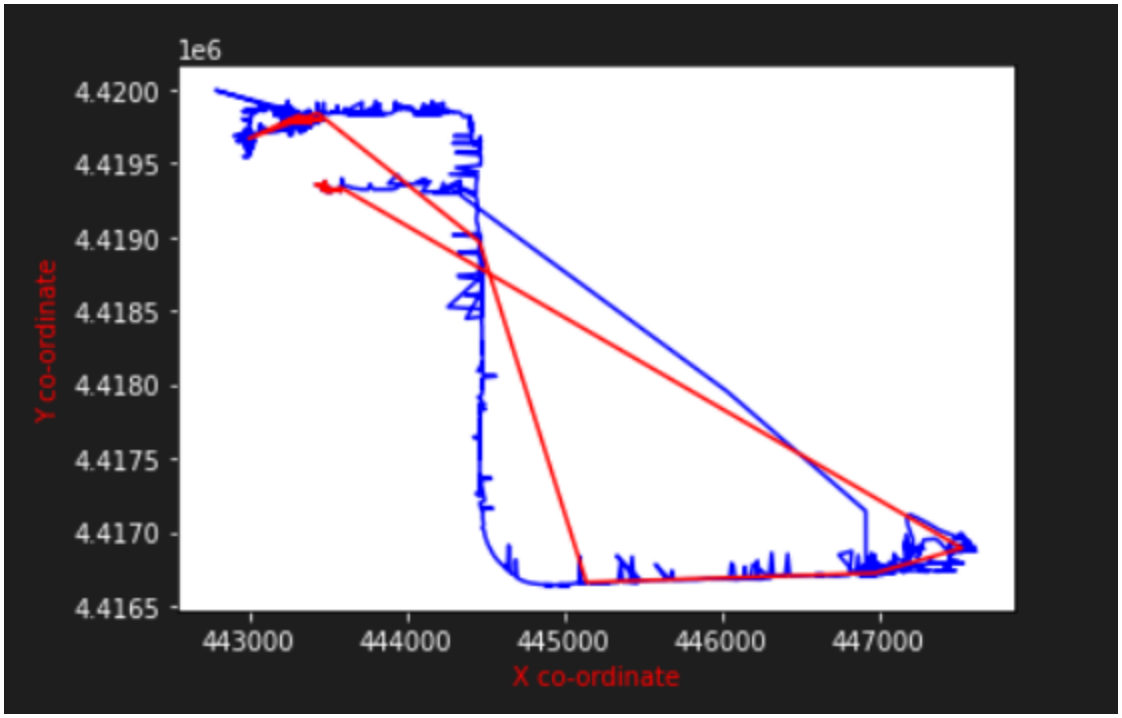


This is the trajectory we obtain for the given user and given day.

5. Some other Plots






1e6
4.4200
4.4195
4.4190
4.4185
4.4180
4.4175
4.4170
4.4165
Y co-ordinate
443000
444000
445000
446000
447000
X co-ordinate

FURTHER PROSPECTS

There is no single way in which we can do imputation. We can create missing values and impute them from the original dataset in multiple ways and techniques.
The data present is very random, which makes the task a little complicated. There is no uniformity in the amount of data present for different users. Moreover, the data for a user is not at all guaranteed to be present at some duration of the day for all the days.

To overcome this issue, we can use the union of all the days.
Say we have User Ui and his data present for days D1, D2, D3…Dk. Now we combine the whole data for User Ui into one single data frame sorted as per the time.

$$U_i = D_1 \coprod D_2 \coprod D_3 \cdots \coprod D_k$$

Ui	TimeSlot1	TimeSlot2	TimeSlot3	TimeSlot4	TimeSlot5	TimeSlot6	TimeSlot7	TimeSlot8	TimeSlot9	TimeSlot10
D1										
D2										
D3										
D4										
D5										
D6										
D7										
D8										
D9										
D10										

Here we can see how randomly the data is present. Say if we need to impute for D8, then data is present only for TimeSlot1. If we create artificial NaN values, we can achieve data imputation, but the results won't be that satisfying since we have very less original data.

So now we can take a union of all the days from D1 to D10 so there is enough data present for a satisfactory imputation result. After the union for TimeSlot1, data is present from D1, D4, D8, and D10.

We take a random sample from this data to get a good Gaussian curve. This sampling can also be weighted as if more data points are present on a particular day, then that day is given higher weight than other days while sampling.
After we get our sample data points for a particular day, say for D4, we need to take data from other days in TimeSlot 2, 4, 5, and 6. Then we use our model and impute the missing values.

The values we get after this imputation would make more sense because we have some data present already.

TRAJECTORY SIMILARITY MEASURES

We can also devise some similarity measures so that we can cluster similar kinds of users. If we don't have data present for a particular user someday, then we can use the data available for other similar users to predict the user's position on that day.

Trajectory similarity measures are used to quantify the similarity or dissimilarity between two or more trajectories, representing objects' paths or movements over time. Here are some commonly used trajectory similarity measures:

1. **Euclidean Distance**: Calculates the straight-line distance between corresponding points in two trajectories. It is a basic measure that considers only the geometric positions of the points.

2. **Dynamic Time Warping (DTW)**: Measures the similarity between two sequences, allowing for differences in the time axis. DTW finds an optimal alignment between two trajectories by warping the time axis to minimize the distance between corresponding points.

3. **Longest Common Subsequence (LCSS)**: Considers the common subsequence between two trajectories, allowing for some degree of tolerance in the spatial and temporal dimensions. It quantifies the similarity by measuring the percentage of common points between the trajectories.

4. **Hausdorff Distance**: Measures the maximum distance between any point in one trajectory and the closest point in the other. It captures both local and global differences between trajectories.

5. **Fréchet Distance**: Determines the minimum leash length needed for a dog to traverse two trajectories simultaneously. It captures both spatial and temporal information and is sensitive to the entire shape of the trajectories.

6. **Edit Distance on Real Sequences (EDR)**: This represents the minimum cost required to transform one trajectory into another using elementary edit operations, such as insertion, deletion, and substitution of points.

7. **Edit Distance with Real Penalty (ERP)**: Similar to EDR, but assigns different costs to different edit operations to reflect their relative importance in the context of the trajectories.

8. **Cosine Similarity**: Measures the cosine of the angle between two trajectory vectors. Trajectories are transformed into feature vectors, and the similarity is computed based on the vector representation.

CONCLUSION

Data imputation is a method for possessing most of the dataset's data and information by substituting missing data with various values.

We did Data Imputation on GPS Trajectory Data. This data is a Time Series Data. We have used the Self-Attention-based Imputation for Time Series (SAITS) model to fill up the missing values in the dataset.

The codebase has a function named ‘impute’ wherein 3 parameters: user number, start day index, and end day index passed. We can change the parameters and get the required results easily. As the code has been made as modular as possible, we can also increase/decrease the frequency of imputation by changing the 30-minute timestamp to a smaller/larger value.

There is huge scope ahead for this project. Various methods of imputation can be tried and tested, similarity measures can be devised so as to group or cluster similar users to predict the behavior of one user from the other similar users.

REFERENCES

[1]
Geolife GPS trajectory dataset - User Guide - Microsoft Research

[2]
https://github.com/WenjieDu/SAITS

[3]
https://arxiv.org/pdf/2202.08516.pdf

[4]
Geolife dataset - heremaps/pptk · GitHub

[5]
https://gis.stackexchange.com/questions/212723/how-can-i-convert-lon-lat-coordinates-to-x-y

[6]
(PDF) Trajectory similarity measures (researchgate.net)